Skin Disease Classification System

AI-Assisted Preliminary Skin Condition Analysis

FRONTEND PROJECT DOCUMENTATION

Author	Fatima Maqbool
Date	June 2026
Repository	https://github.com/your-username/skin-disease-classification-system
Version	1.0 (Beta)

Table of Contents

1. Project Overview

The **Skin Disease Classification System** is a browser-based, AI-assisted screening tool that gives users a fast, preliminary read on common dermatological conditions from a single photograph. A user uploads or drags a close-up image of the affected skin area, the application runs an image classification pass, and within seconds it returns a ranked, confidence-scored prediction together with a short explanation of the condition and recommended next steps.

The interface is intentionally simple — a single guided, three-step flow (upload, analyze, review results) wrapped in a calm, dark, purple-accented visual theme that feels clinical without being cold. The product is explicitly framed as an **educational/informational screening aid**, not a diagnostic device, and a medical disclaimer is surfaced prominently on every page.

Purpose & Objectives

- Give users a quick, low-friction way to get an informed first impression of a visible skin concern before deciding whether to see a dermatologist.
- Demonstrate an end-to-end computer-vision classification pipeline (data preparation, model training, and a consumable front-end) for common dermatological conditions.
- Present results transparently, including a full probability breakdown across all candidate conditions rather than a single opaque label.
- Keep the experience private by design — image handling is scoped to the browser session rather than being persisted on a server.

Target Users

- Individuals who notice a new or changing skin lesion, rash, or mark and want a fast, preliminary indication of what it might be.
- Students and self-learners exploring applied computer vision / dermatology-adjacent machine learning use cases.
- Reviewers, recruiters, and portfolio visitors evaluating the project as a demonstration of front-end and applied ML skills.

Key Features

- Drag-and-drop or click-to-browse image upload with live preview and file validation.
- Single-click "Analyze Image" action that triggers the classification step.
- Confidence-ranked predicted condition with a clear high/medium/low risk badge.
- Full probability distribution across all nine supported skin condition classes.
- A plain-language "About this condition" summary and a recommended next action for the top prediction.
- Persistent, prominent medical disclaimer in the footer of every screen.

2. Technology Stack

The front end is intentionally built with framework-free, dependency-light web fundamentals so that it is easy to host as a static site (including directly from a GitHub repository / GitHub Pages) and easy for reviewers to read end-to-end.

Markup	Semantic HTML5
Styling	Hand-written CSS3 (custom properties / CSS variables for the dark purple theme, Flexbox & CSS Grid for layout, no external CSS framework)
Scripting	Vanilla JavaScript (ES6+) — DOM manipulation, drag-and-drop events, file handling, and rendering of results; no front-end framework (React/Vue/Angular) is used
State management	Local, in-memory JavaScript state (no Redux/Context store needed for this scope)
Model / inference	Image classification logic backed by computer-vision models prototyped in a companion Jupyter/Colab notebook (see Section 4). The shipped front end keeps inference scoped to the browser session — no image data is uploaded to a server.
Fonts & Icons	System / web-safe font stack; lightweight inline SVG icons for upload, lock, chart and status indicators
Tooling	Standard browser dev tools; no build step or bundler is required to run the app

Companion Machine Learning Stack (Notebook)

The classification approach behind the UI was developed and benchmarked in a separate Python notebook (`CV_PROJECT.ipynb`), which is included in the repository alongside the front end for transparency and reproducibility. Two complementary approaches were explored:

Approach	Details
Transfer-learning CNN	MobileNetV2 (ImageNet weights, frozen base) + Global Average Pooling + Dense(128, ReLU) + Dropout(0.3) + Dense(9, Softmax). Trained with Keras `ImageDataGenerator` augmentation (rotation, zoom, shear, horizontal flip) for 15 epochs.
Bag of Visual Words (BoVW)	Dense-patch feature extraction via a frozen ResNet50 backbone, 1,200-word visual vocabulary built with MiniBatchKMeans, classified with an RBF-kernel SVM (class-balanced).

Approach	Details
Additional experiment	A VGG16 transfer-learning baseline trained on CIFAR-10 was used separately to validate the transfer-learning pipeline mechanics.

The dataset used for training ("Split_smol") contains 9 dermatological classes — Actinic keratosis, Atopic Dermatitis, Benign keratosis, Dermatofibroma, Melanocytic nevus, Melanoma, Squamous cell carcinoma, Tinea Ringworm Candidiasis, and Vascular lesion — split into 697 training and 181 validation images. The BoVW + SVM pipeline reached roughly **72.9% test accuracy** across the 9 classes on a 140-image hold-out set; per-class performance and a full classification report are recorded in the notebook.

Note: the production front end mirrors this 9-class taxonomy in its results UI (predicted condition, confidence score, and full probability breakdown). Teams adopting this project should confirm which trained model artifact is wired into `script.js` *for live inference versus used for offline evaluation only.*

3. Project Structure

The project follows a flat, static-site-friendly structure so that it can be opened directly in a browser or deployed to any static host without a build pipeline.

```
skin-disease-classification-system/
■
■■■ index.html              # Single-page app shell: header, hero, upload & results sections
■■■ style.css                # Theme variables, layout, components, responsive rules
■■■ script.js                # Upload handling, drag-and-drop, validation, classification fl
ow, results rendering
■■■ assets/
■   ■■■ icons/               # Inline/standalone SVG icons (upload, lock, chart, etc.)
■   ■■■ images/              # Static imagery used in the UI (preview placeholders, brandin
g)
■■■ notebooks/
■   ■■■ CV_PROJECT.ipynb     # Model exploration: CNN (MobileNetV2) + BoVW/SVM training & e
valuation
■■■ docs/
■   ■■■ Skin_Disease_Classifier_Documentation.pdf   # This document
■■■ LICENSE
■■■ README.md                # Quick-start, setup, and usage instructions
```

Directory & File Purpose

Path	Purpose
`index.html`	Defines the single-page structure: branding header, hero/intro panel, the numbered upload step, the run-classification step, and the results panel.
`style.css`	Owns the dark purple visual theme via CSS custom properties, responsive grid/flex layout, card and badge components, and state styles (drag-over, success toast, disabled buttons).
`script.js`	Implements file selection & drag-and-drop handlers, client-side validation (type/size/dimensions), the analyze action, and rendering of the prediction card and probability list.
`assets/`	Holds icons and any static images referenced by the UI.
`notebooks/CV_PROJECT.ipynb`	Documents the data loading, CNN training, and BoVW/SVM experimentation that informs the classification logic.
`README.md`	Project summary plus install/run instructions for new contributors.

4. Application Screenshots

This section walks through every screen state of the application in the order a user naturally encounters them, from first landing on the page through to reviewing a classification result.

Screen 1 — Home / Landing View

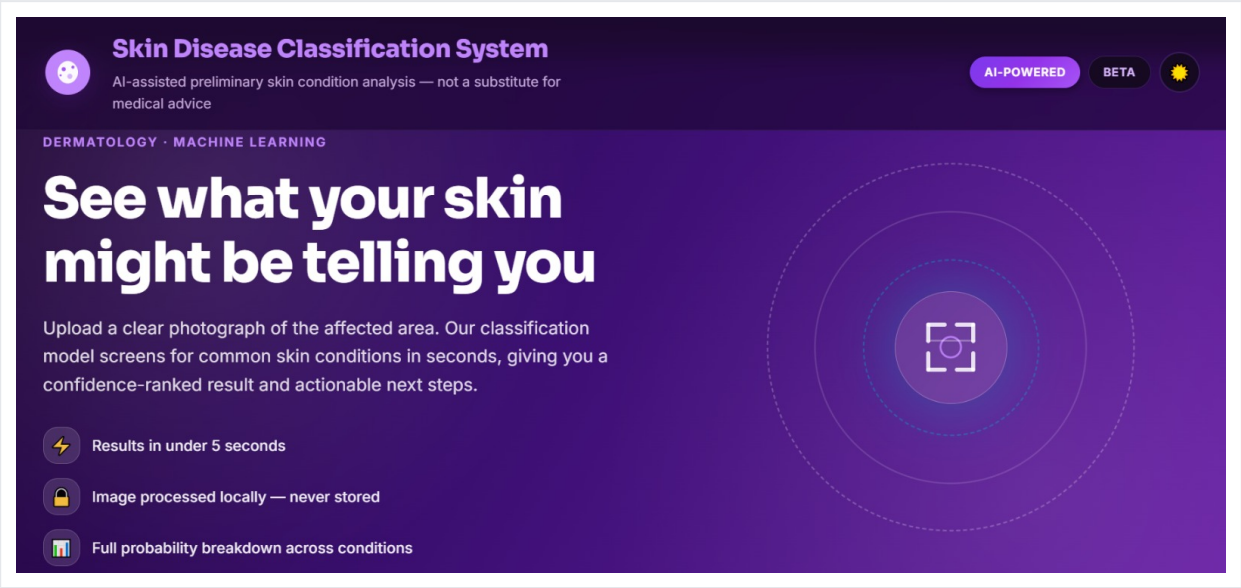

Figure 1. Landing hero section with headline, value propositions, and the scan iconography.

Purpose: Orient the user, build trust, and set expectations before they commit to uploading a personal photo.

Features on this screen:

- Product name and one-line positioning statement in the top header bar, alongside "AI-Powered" and "Beta" status badges.
- Headline ("See what your skin might be telling you") and supporting copy explaining the upload-to-result flow.
- Three trust-building highlights: results in under 5 seconds, local-only image processing, and a full probability breakdown.
- A decorative animated scan/target icon reinforcing the "analysis" concept.

User interactions: Purely informational — the user scrolls past this section toward the upload step; no inputs are required here.

Design considerations: A bold purple gradient panel separates this orientation section from the darker, more functional sections below it, giving the page a clear visual hierarchy from "marketing" to "tool."

Components used: App header / nav bar, badge pill components, icon + text feature list, hero illustration.

Responsive behavior: On tablet and mobile, the two-column hero (copy + illustration) stacks vertically, the illustration scales down, and the feature list remains single-column.

Screen 2 — Upload Your Image (Empty State)

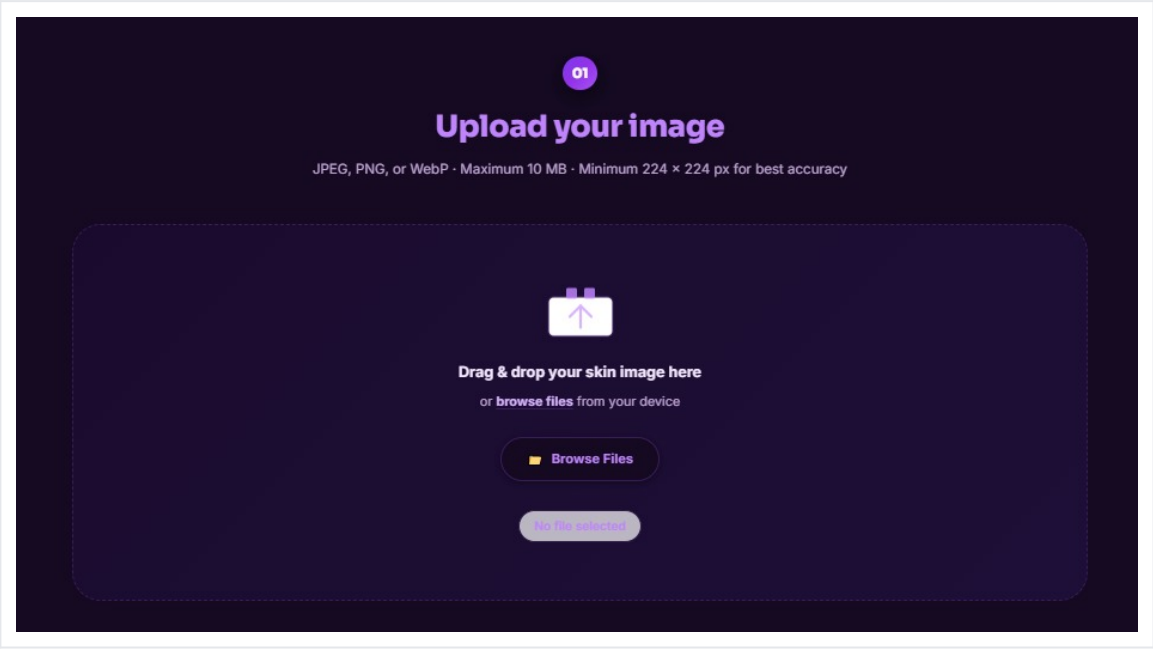

Figure 2. Step 01 — the empty drag-and-drop upload zone with file constraints.

Purpose: Collect a clear image of the affected skin area from the user.

Features on this screen:

- Numbered step indicator ("01") and heading to anchor the user's place in the flow.
- File requirement hints: accepted formats (JPEG, PNG, WebP), maximum size (10 MB), and minimum resolution (224×224 px).
- Dashed drop-zone supporting both drag-and-drop and a "Browse Files" button.
- A status pill confirming the current selection state ("No file selected").

User interactions: Drag a file onto the zone, or click "Browse Files" / the "browse files" inline link to open the native file picker.

Design considerations: The dashed border and upload glyph are universally recognizable upload affordances; constraints are surfaced up front to reduce failed uploads later in the flow.

Components used: Step badge, drop-zone card, primary/secondary buttons, status pill.

Responsive behavior: The drop-zone retains generous tap targets on mobile; drag-and-drop gracefully degrades to tap-to-browse on touch devices.

Screen 3 — Image Selected & Preview

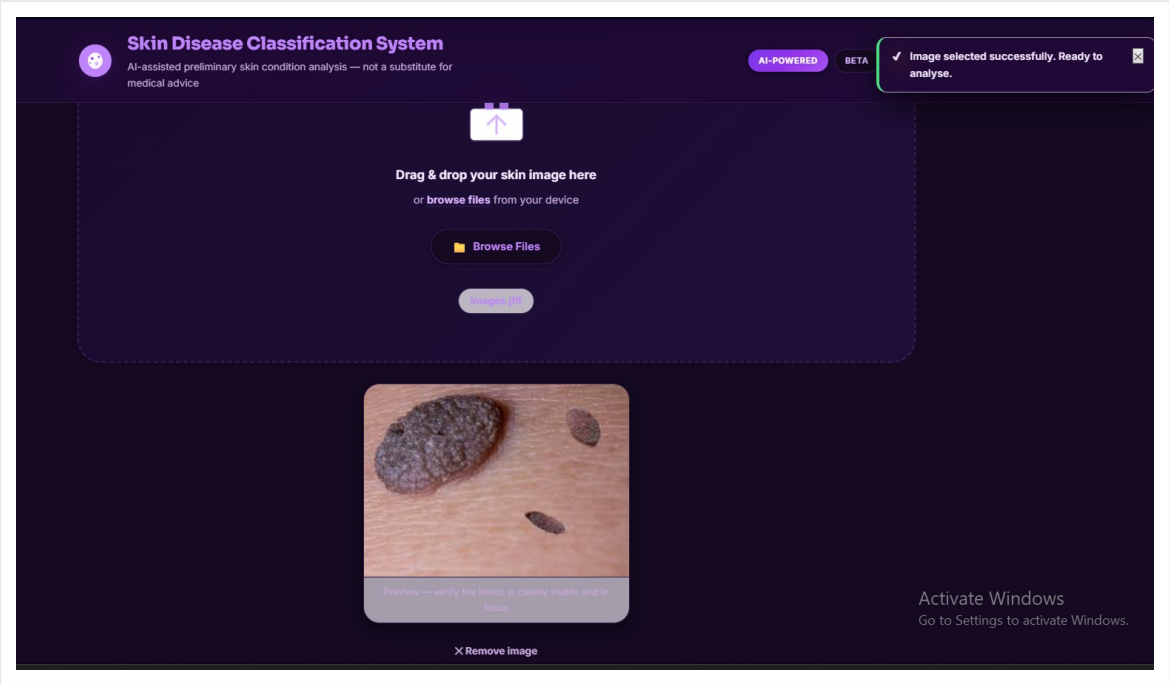

Figure 3. Selected image preview with a success toast confirming the file is ready to analyze.

Purpose: Let the user confirm the right photo was selected, and that the lesion is clearly visible, before running the analysis.

Features on this screen:

- Inline image preview rendered directly from the selected file.
- Guidance caption: "Preview — verify the lesion is clearly visible and in focus."
- "Remove image" action to clear the selection and start over.
- A success toast notification ("Image selected successfully. Ready to analyse.") confirming validation passed.

User interactions: Review the preview, optionally remove and re-select an image, or proceed to the analyze step.

Design considerations: Immediate visual feedback (preview + toast) closes the loop on the upload action so the user isn't left wondering whether the upload worked.

Components used: Image preview card, dismissible toast/notification component, text link action ("Remove image").

Responsive behavior: The preview image scales to the available width; the toast repositions to avoid overlapping the header on narrow viewports.

Screen 4 — Run Classification

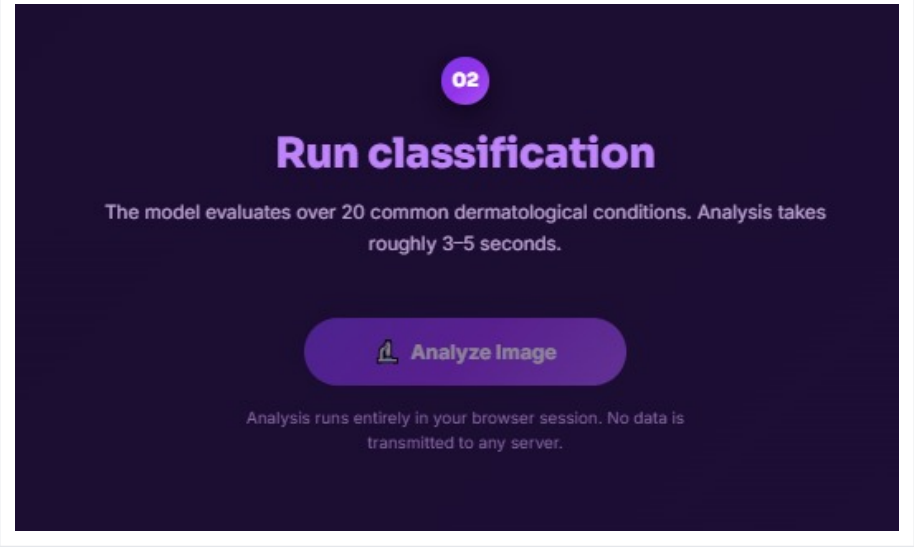

Figure 4. Step 02 — the Analyze Image action that triggers the classification pass.

Purpose: Give the user an explicit, deliberate action to start the analysis, and set expectations for how the model works and how long it will take.

Features on this screen:

- Numbered step indicator ("02") with the "Run classification" heading.
- Explanatory copy: the model evaluates over 20 common dermatological conditions, typically in 3–5 seconds.
- Primary "Analyze Image" call-to-action button with a scan icon.
- Reassurance copy that analysis runs entirely in the browser session and no data is transmitted to a server.

User interactions: Click "Analyze Image" to trigger the classification routine; the button enters a loading/disabled state while processing.

Design considerations: Separating "upload" and "analyze" into two distinct, numbered steps avoids surprising the user with an automatic analysis the moment a file is chosen, and reinforces the privacy-by-design messaging from the hero section.

Components used: Step badge, primary CTA button, helper/disclaimer text.

Responsive behavior: The CTA remains centered and full-width-friendly on small screens; helper copy wraps to two lines rather than truncating.

Screen 5 — Classification Results

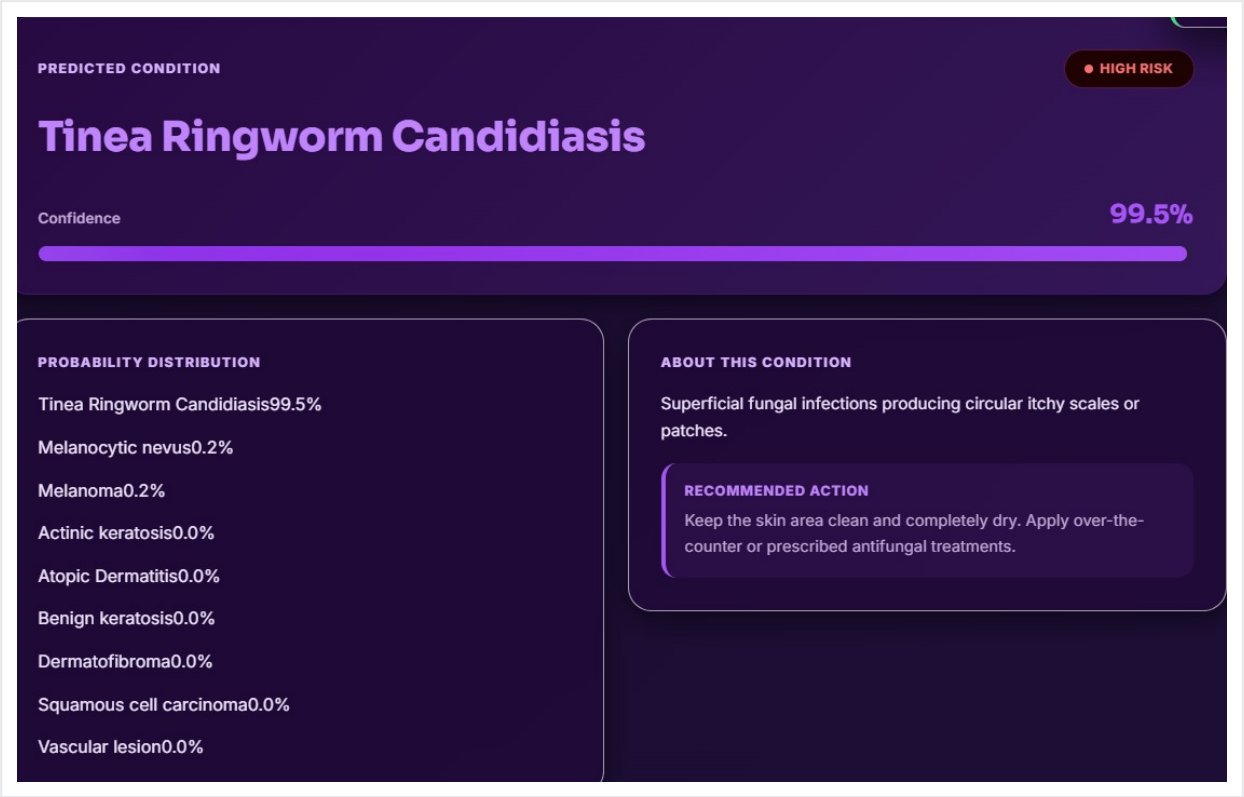

Figure 5. Predicted condition card with confidence score, full probability distribution, and recommended action.

Purpose: Communicate the model's prediction clearly, transparently, and with appropriate context so the user can make an informed decision about next steps.

Features on this screen:

- Predicted condition name in large type, paired with a color-coded risk badge (e.g. "High Risk").
- A confidence percentage with an accompanying progress bar visualization.
- Full probability distribution listing every candidate class and its score, so users can see what else the model considered.
- An "About this condition" panel with a plain-language description.
- A "Recommended action" callout with practical next-step guidance.

User interactions: Scan the headline prediction, optionally scroll the full probability list, and read the recommended action; the user can scroll back up to analyze a different image.

Design considerations: Showing the complete probability breakdown — not just the top label — is a deliberate transparency choice that helps prevent users from over-trusting a single confident-looking number, especially for visually similar conditions.

Components used: Result header card, risk badge, progress/confidence bar, two-column info panel (probability list + condition summary), recommended-action callout box.

Responsive behavior: The two-column results layout (probability distribution + about/recommendation) collapses to a single stacked column on tablet and mobile widths.

Screen 6 — Medical Disclaimer & Footer

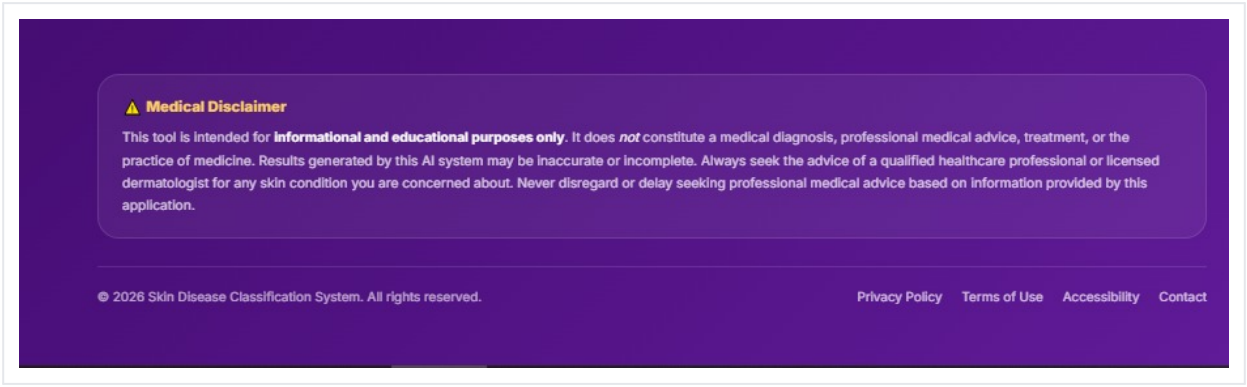

Figure 6. Persistent medical disclaimer and footer navigation, shown on every page.

Purpose: Set legal and ethical expectations clearly: the tool is informational only and must never replace professional medical advice.

Features on this screen:

- A clearly bordered, color-flagged disclaimer panel with a warning icon.
- Explicit language stating the tool does not constitute a medical diagnosis and results may be inaccurate or incomplete.
- Footer copyright line and secondary navigation links (Privacy Policy, Terms of Use, Accessibility, Contact).

User interactions: Read-only; links route to their respective informational pages.

Design considerations: Persisting this disclaimer on every screen — not just on first load — ensures the disclosure stays visible right next to where results are shown.

Components used: Alert/callout box, footer link list.

Responsive behavior: Footer links wrap into a centered stack on narrow viewports; the disclaimer box retains full readability at all widths.

Screen 7 — Full Page Overview

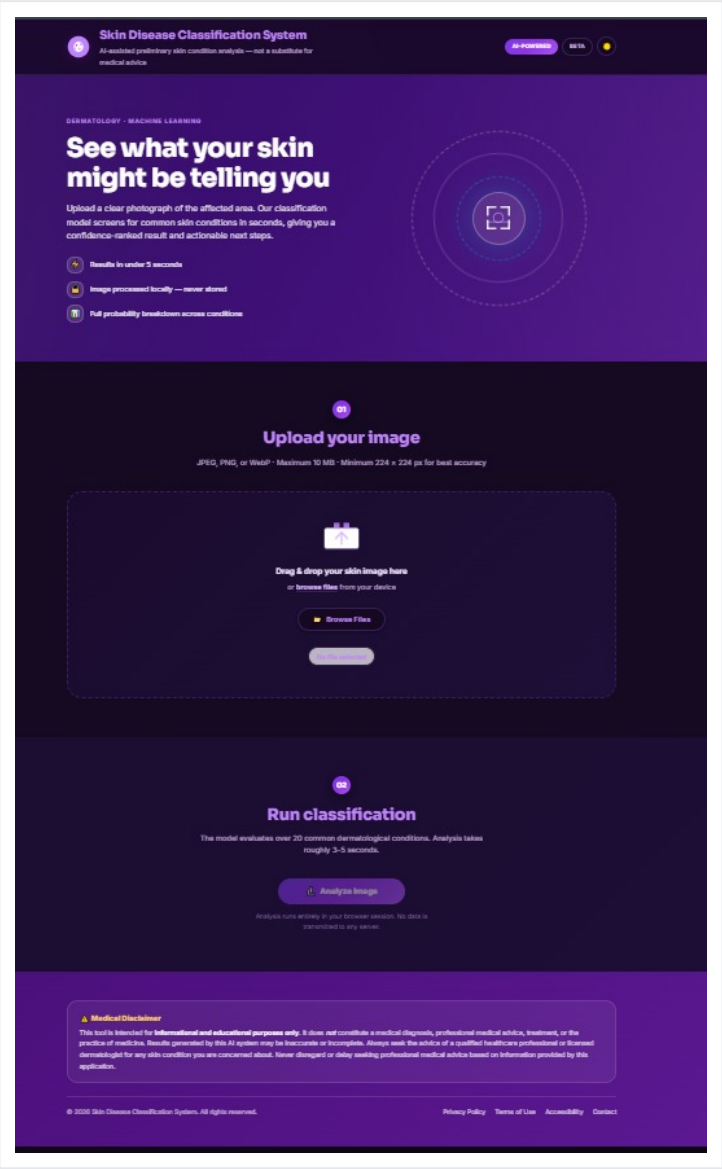

Figure 7. End-to-end scroll view showing how all sections compose into a single guided page.

Purpose: Illustrate how the hero, upload, run-classification, and disclaimer sections compose into one continuous, single-page application rather than separate routes.

Navigation flow: Hero/intro → Upload (step 01) → Run classification (step 02) → Results (rendered in place once analysis completes) → persistent disclaimer/footer.

5. UI/UX Design Details

Color Palette

The interface uses a dark, near-black backdrop with a vivid purple/violet accent family, evoking a clinical-but-modern, "AI lab" feel.

- `#1A0F2E` — Background (deep aubergine/black)
- `#3B0764` — Hero panel / dark purple
- `#7C3AED` — Primary accent (CTAs, links, headings)
- `#A78BFA` — Secondary accent / icon highlights
- `#FFFFFF` — Primary text on dark surfaces
- `#F59E0B` — Status / warning accents (disclaimer)

Typography

A clean, rounded sans-serif typeface family is used throughout: bold, large-scale headings for the hero and result headline, medium-weight section titles, and a regular weight for body copy, with a slightly muted/lighter tone used for secondary helper text (file constraints, captions, footer links).

Icons

Lightweight line/glyph icons (upload arrow, lock, bar-chart, scan/target, warning triangle, checkmark) are used consistently to reinforce meaning at a glance — e.g. the lock icon next to "Image processed locally" reinforces the privacy claim without requiring the user to read it.

Design System

- Reusable badge/pill component for status labels ("AI-Powered", "Beta", risk levels).
- Card components with consistent border-radius and padding for the upload zone, preview, and result panels.
- A single primary-button style (filled purple, rounded) used for the main call-to-action across steps.
- Numbered step badges (01, 02) to communicate progress through a linear flow.

Responsive Design Strategy

The layout is built mobile-first with CSS Flexbox/Grid: multi-column sections (hero copy + illustration, results probability list + about panel) collapse into a single column below tablet breakpoints, while spacing, font sizes, and tap targets scale down proportionally to remain comfortable on small screens.

Accessibility Considerations

- High-contrast white/light text on dark purple backgrounds to support legibility.
- Descriptive alt text recommended for all uploaded-image previews and icon-only buttons.

- Drag-and-drop is paired with an equivalent keyboard- and screen-reader-accessible "Browse Files" button.
- Status changes (file selected, analysis complete) are surfaced as visible toast messages, recommended to also be exposed via an ARIA live region for assistive technology.

6. Component Documentation

Although the project is built with vanilla HTML/CSS/JS rather than a component framework, the UI is composed of clearly separable, reusable building blocks. The table below documents each as if it were a component contract, to guide any future migration to a component-based framework.

Component	Purpose / Props / Reusability
AppHeader	**Purpose:** Branding, product name/tagline, status badges. **Props:** title, subtitle, badges[]. **Reusable** across every page/view.
UploadDropzone	**Purpose:** Accepts drag-and-drop or browsed files; validates type/size/dimensions. **State:** selectedFile, isDragActive, error. **Props:** accept[], maxSizeMB, minDimensions. **Reusable** for any future multi-image or batch-upload feature.
ImagePreviewCard	**Purpose:** Renders the selected image with a caption and a remove action. **Props:** imageSrc, caption, onRemove. **State:** none (controlled by parent).
Toast / Notification	**Purpose:** Transient confirmation/status messages (e.g. "Image selected successfully"). **Props:** message, variant (success/error/info), dismissible. **Reusable** across all user actions requiring feedback.
AnalyzeButton	**Purpose:** Primary CTA to trigger classification. **Props:** label, icon, isLoading, onClick. **State:** disabled while a request/inference is in progress.
ResultHeaderCard	**Purpose:** Shows predicted condition, risk badge, and confidence bar. **Props:** conditionName, riskLevel, confidencePercent.
ProbabilityList	**Purpose:** Renders the ranked list of all class probabilities. **Props:** predictions: Array<{label, score}>.
AboutConditionPanel	**Purpose:** Plain-language description plus a recommended-action callout. **Props:** description, recommendedAction.
DisclaimerBanner	**Purpose:** Persistent legal/medical disclaimer. **Props:** none (static content). **Reusable** on every route/page.
AppFooter	**Purpose:** Copyright and secondary navigation links. **Props:** year, links[].

7. Features and Functionality

Authentication

The current build is a single-user, no-login utility — there is no authentication layer, consistent with its privacy-by-design positioning (images are processed client-side and never associated with an account). A future authenticated mode could support saving scan history per user.

Navigation

Navigation is a single linear, scroll-based flow rather than multi-page routing: hero → upload → analyze → results, all within one HTML document. Footer links to Privacy Policy, Terms of Use, Accessibility, and Contact are placeholders for secondary informational pages.

Forms / Input Handling

The only form-like input is the file selector. Client-side validation checks file type (JPEG/PNG/WebP), maximum size (10 MB), and minimum resolution (224×224 px) before allowing the user to proceed, surfacing clear inline guidance and a success toast on valid selection.

API / Model Integration

Per the in-app messaging ("Analysis runs entirely in your browser session. No data is transmitted to any server."), inference is scoped to the client rather than calling a remote prediction API. The companion notebook documents the model-training side of this pipeline (MobileNetV2 transfer-learning CNN and a BoVW + SVM baseline, see Section 2); teams extending this project to call a hosted model API should add a documented REST/JSON contract (e.g. `POST /predict` with an image payload, returning a label and probability array) and update this section accordingly.

Search / Filtering

Not applicable in the current scope — the app handles a single image per analysis run rather than browsing or filtering a dataset.

Dashboard Functionality

The results panel functions as a lightweight, single-result dashboard: predicted label, risk level, confidence score, full probability breakdown, and contextual guidance, all rendered together so the user does not need to navigate elsewhere to interpret the output.

Custom Features

- Full transparency probability breakdown (not just the top-1 prediction).
- Explicit, persistent medical disclaimer pattern, present on first load and after results render.
- Privacy-first messaging baked directly into the analyze step's helper copy.

8. Workflow and User Journey

The end-to-end journey is designed to be completed in well under a minute, with each step giving the user a clear sense of progress and control.

- **1. Arrive:** User lands on the hero section and reads the value proposition and trust signals (speed, local processing, full breakdown).
- **2. Upload:** User drags an image onto the drop-zone or clicks "Browse Files" to select a photo of the affected skin area.
- **3. Confirm:** The app renders an inline preview and a success toast; the user verifies the lesion is clearly visible, optionally removing and re-selecting the image.
- **4. Analyze:** User clicks "Analyze Image"; the app runs the classification pass (≈3–5 seconds) while the button shows a loading/disabled state.
- **5. Review results:** The predicted condition, risk badge, confidence score, full probability list, condition summary, and recommended action are rendered.
- **6. Act:** User reads the persistent medical disclaimer and decides on next steps — e.g. seeking professional dermatological advice — or scrolls back up to analyze another image.

9. Performance Optimization

Because the application is a static, dependency-light page, performance is governed primarily by asset weight and client-side image handling rather than network round-trips.

- **No framework overhead:** shipping plain HTML/CSS/JS avoids the parse/hydrate cost of a front-end framework runtime.
- **Local image handling:** using the File/Blob APIs and object URLs to preview images avoids any network upload before analysis, removing latency from that step entirely.
- **Lazy / on-demand rendering:** the results panel and its sub-components are only constructed once an analysis completes, keeping the initial page light.
- **Image optimization:** compressing and right-sizing any static brand/illustration assets (e.g. served as WebP where supported) keeps the hero section fast on first paint.
- **Caching:** static assets (HTML/CSS/JS, icons) are cacheable by the browser/CDN since the deployment is a static site with no per-user dynamic content.
- **Debounced/guarded actions:** the "Analyze Image" button is disabled while a run is in progress to avoid duplicate, redundant classification passes.

10. Challenges and Solutions

Challenge	Solution / Lesson Learned
Building user trust for an AI health-adjacent tool without a backend.	Leaned heavily on transparent UI patterns — visible confidence scores, full probability breakdowns, and a persistent, clearly worded medical disclaimer — instead of presenting a single opaque "answer."
Keeping the experience fast without a remote inference API.	Scoped the demo to in-browser-session messaging and a guided two-step (upload → analyze) flow, so users always understand what is happening and when results are ready.
Limited & imbalanced training data across 9 skin condition classes (697 train / 181 val images, with class counts ranging from 56–81 images).	Used transfer learning (frozen MobileNetV2 / ResNet50 backbones) plus data augmentation to make the most of a small dataset, and used class-balanced SVM weighting for the BoVW baseline.
Comparing very different modeling approaches (deep CNN vs. classical BoVW + SVM) fairly.	Evaluated both with the same train/validation split and full classification reports (precision/recall/F1 per class), documented directly in the notebook for traceability.
Communicating model uncertainty responsibly.	Chose to always render the full probability distribution rather than only the top prediction, and to badge risk level alongside the label, reducing the chance of users over-trusting a single confident-sounding output.

11. Future Improvements

- Wire a production-grade, versioned model artifact (e.g. an exported TensorFlow.js graph or a hosted inference API) into the live "Analyze Image" action.
- Expand and rebalance the training dataset per class to improve accuracy and reduce variance for under-represented conditions (e.g. Tinea Ringworm Candidiasis, which had only 56 training images).
- Add user accounts and a private scan history dashboard for users who want to track changes over time.
- Surface model explainability (e.g. a Grad-CAM heatmap overlay) so users can see which region of the image most influenced the prediction.
- Add multi-language support and localized medical disclaimers.
- Introduce automated accessibility and visual-regression testing as part of CI.
- Add a downloadable/printable PDF summary of an individual scan result for sharing with a healthcare provider.

12. Installation and Setup

Prerequisites

- A modern web browser (Chrome, Firefox, Edge, or Safari, latest two versions).
- Git, to clone the repository.
- Optionally, a simple static file server (e.g. the VS Code "Live Server" extension, or Python's built-in HTTP server) for local development.
- Optionally, Python 3.9+ with Jupyter/Colab access if you intend to re-run or extend the model-training notebook.

Installation Steps

1. Clone the repository:

```
git clone https://github.com/your-username/skin-disease-classification-system.git
cd skin-disease-classification-system
```

2. Open the app directly, or serve it locally:

```
# Option A — open directly
open index.html

# Option B — serve locally (recommended)
python3 -m http.server 5500
# then visit http://localhost:5500
```

Environment Variables

The current static front end requires no environment variables or secrets to run. If a hosted inference API is added in the future, document its base URL and any API key as, for example, `API_BASE_URL` and `API_KEY` in a local, git-ignored `.env` file.

Run Commands

```
python3 -m http.server 5500
```

Build Commands

No build step is required — the project ships plain HTML/CSS/JS that runs directly in the browser. If a future iteration introduces a bundler or framework, document the `install` and `build` commands here.

Running the Notebook

Open `notebooks/CV_PROJECT.ipynb` in Google Colab or Jupyter, mount/point it at the skin-disease dataset, and run the cells sequentially to reproduce the CNN and BoVW + SVM training and evaluation runs.

13. Conclusion

The Skin Disease Classification System demonstrates a complete, thoughtfully designed path from a real computer-vision problem to an approachable, trustworthy front-end experience. The project pairs a clean, framework-free HTML/CSS/JS implementation with a well-documented machine-learning exploration phase — comparing a MobileNetV2 transfer-learning CNN against a classical Bag-of-Visual-Words + SVM pipeline across nine dermatological conditions — and translates that work into a simple three-step user flow: upload, analyze, and review.

Its strongest design decisions are also its most responsible ones: surfacing the full probability distribution instead of a single label, keeping image handling local to the browser session, and pairing every result with a clear, persistent medical disclaimer. Together these choices make the tool genuinely useful as an educational screening aid while staying honest about its limits.

With the future improvements outlined in Section 11 — particularly wiring in a production inference path and expanding the training dataset — this project has a clear runway from a strong portfolio piece to a more fully realized product.